

VideoPrism 실험: Loss 함수별 성능 비교 (K400 & SSv2)

1. 실험 환경

- 데이터셋 1: Kinetics-400

(비디오 수: 40,000개 (66GB), 클래스: 400개, Task: 일반 행동 인식 (수영, 농구 등))

- 데이터셋 2: Something-Something V2 (SSv2)

(비디오 수: 168,913개 (19GB), 클래스: 174개, Task: 시간적 행동 인식 (물건을 왼쪽→오른쪽으로 이동 등))

- 모델: VideoPrism-Base (Frozen Backbone) + Linear Classifier

- 하드웨어: Google Cloud TPU v4-8

- 학습 스텝: **5,000 스텝** × 4가지 실험

지난 1차 실험 결과 분석을 통해 도출된 한계점을 극복하고 가설을 검증하기 위해 두 가지 방향의 추가 실험을 진행했습니다. 첫째는 절대적인 학습량 부족을 해결하기 위해 K400 데이터셋의 학습 스텝을 기존 5배로 늘린 심화 실험이며, 둘째는 시간적 순서가 분류에 미치는 영향을 확인하기 위해 행동 중심 데이터셋인 Something-Something V2(SSv2)를 도입한 신규 실험입니다.

실험은 Google TRC(TPU Research Cloud) 프로그램을 통해 지원받은 Cloud TPU v4-8 노드를 활용하여 진행되었습니다. 데이터셋의 경우 Kinetics-400 전체 데이터(약 24만 개)를 모두 사용하는 것은 100GB라는 스토리지 상의 제약이 있어, 전체의 약 16%에 해당하는 40,000개의 비디오(약 66GB)를 선별하여 Mini-Kinetics 데이터셋을 직접 구축하였습니다. Ssv2의 경우 전체 데이터셋의 용량이 19GB 정도 밖에 되지 않아 168,913개의 영상 데이터셋을 모두 다운받을 수 있었습니다.

```
>>> Starting Experiment: Label Smoothing
Loading VideoPrism backbone...
[Info] Dataset initialized with 40000 videos.
[Step 0000] Loss: 6.0148 | Accuracy: 12.50%
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x5616f464c500] mov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x56170314b180] mov atom not found

3 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log

*** System restart required ***
Last login: Sun Feb 22 17:15:03 2026 from 35.235.243.194
(base) zookavincan@tlv-n-bb74ball-w-0:~$ python ssv22.py
=====
VideoPrism Fine-tuning on SSv2 (Temporal Task)
- Fixed Cosine Distance SoftDTW applied -
=====
Loading label files...
[Info] Loaded 168,913 video labels.

>>> Starting Experiment: SoftDTW (Temporal Consistency)
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Scanning video directory...
[Info] Found 168,913 valid videos for training.
[Step 0000] Loss: 5.2392 | Accuracy: 12.50%
```

2. 실험 결과

(1) K-400 심화 실험 결과 및 분석 (학습량 증가)*

기존 1,000 Step에서 5,000 Step으로 학습량을 5배 증가시켜 도출된 K400의 각 케이스별 정확도와 손실값은 아래 표와 같습니다.

K400	Loss 함수	Accuracy
1	Original (Cross Entropy)	64.00%
2	Original + Label Smoothing	66.12%
3	Original + SoftDTW	64.38%
4	Original + Label Smoothing + SoftDTW	66.88%

실험 결과, 베이스라인(Original) 정확도가 기존 47.25%에서 64.00%로 무려 16.75% 포인트 상승했습니다. 이는 지난 실험에서 보였던 낮은 성능이 절대적인 학습량 부족 때문이었음을 증명할 수 있습니다. 또한, Label Smoothing을 적용한 경우 여전히 단독 성능이 우수(66.12%)했으며, 이번 학습에서는 Label Smoothing과 SoftDTW를 함께 적용했을 때 66.88%로 가장 높은 성능을 달성했습니다. 이는 충분한 학습이 이루어질 경우, 과적합을 방지하는 Smoothing 기법과 시계열적 특징을 부여하는 SoftDTW가 시너지를 낼 수 있음을 알 수 있습니다.

```
[Step 04700/10000] Loss: 1.6226 | Acc: 87.50% | ETA: 352.5분
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e04af2000] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556cd5cfac80] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e01575180] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556eed5bea00] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556eedb69900] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556ee6ea6f00] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e0419d180] moov atom not found
[Step 04800/10000] Loss: 2.3096 | Acc: 62.50% | ETA: 345.8분
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556eed5c5900] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e0e511180] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e01575400] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e0419d680] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556cefed0c80] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556cefed0280] moov atom not found
[Step 04900/10000] Loss: 3.7098 | Acc: 25.00% | ETA: 339.3분
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556eed5be280] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e0afb8500] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e0ccbdb80] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556df3d47400] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556e0cff0f00] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556eed5bf180] moov atom not found
[mov,mp4,m4a,3gp,3g2,mj2 @ 0x556eeaad2a00] moov atom not found
실험 결과 저장됨 : /home/zoekavincan/k400_mini/logs/results.json

[✓] 실험 완료 : Smoothing + SoftDTW
  최종 Loss: 430.7003
  최종 Accuracy: 66.88%
  총 소요 시간: 330.1분

=====
🚀 실험 완료 !
📁 로그 위치 : /home/zoekavincan/k400_mini/logs
📁 체크포인트 위치 : /home/zoekavincan/k400_mini/checkpoints
=====
(base) zoekavincan@t1v-n-bb74ball-w-0:~/k400_mini/annotations$
```

(2) Something-Something V2 실험 결과 및 분석 (데이터셋 특성 비교)

K400(객체 중심)에서 SoftDTW의 단독 효과가 미미했던 이유가 '데이터셋의 특성' 때문이라는 가설을 검증하기 위해, 프레임의 순서가 결정적인 역할을 하는 SSv2(행동 중심) 데이터셋으로 동일한 4가지 Loss 조합을 5,000 Step 학습하여 다음과 같은 결과를 얻었습니다.

SSv2	Loss 함수	Accuracy
1	Original (Cross Entropy)	30.38%
2	Original + Label Smoothing	31.13%
3	Original + SoftDTW	30.75%
4	Original + Label Smoothing + SoftDTW	29.50%

SSv2는 클래스 수가 174개이며 미세한 행동 변화(예: 왼쪽으로 밀기 vs 오른쪽으로 밀기)를 구분해야 하는 고난도 데이터셋이므로 전체적인 베이스라인이 약 30%대로 낮게 형성되었습니다.

결과를 분석해보면, 예상대로 Label Smoothing을 적용했을 때(31.13%) 가장 안정적으로 최고 성능을 기록했습니다. 주목할 점은 SoftDTW 단독 사용 시(30.75%) Original 대비 소폭(0.37%p) 성능 향상을 보였다는 것입니다. 이는 K400에서는 나타나지 않았던 현상으로, 영상의 시간적 흐름이 중요한 SSv2 데이터셋의 특성상 시계열 정렬을 강제하는 SoftDTW가 유의미하게 작용하기 시작했음을 보여줍니다.

다만 두 기법을 혼합했을 때는 오히려 성능이 하락했는데, 고난도 데이터셋에서 두 가지 서로 다른 방향의 Loss 규제가 모델의 학습을 방해했거나, Loss 간의 가중치(현재 0.1) 튜닝이 추가로 필요한 것 같습니다.

```
ssh.cloud.google.com/v2/ssh/projects/project-56114b1c-2887-459c-b4a/locations/us-central2/b/nodes/node-27authuser-08hl=ko&projectNumber=1031183227159&useAdminProxy=true&pageViewId=C484FE83-7D8A-474E-9DDA-9333E7A6969A ...
ssh.cloud.google.com/v2/ssh/projects/project-56114b1c-2887-459c-b4a/locations/us-central2-b/nodes/node-27authuser-08hl=ko&projectNumber=1031183227159&useAdminProxy=true&pageViewId=C484FE83-7...
브라우저에서 SSH를 통해 연결
파일 열람 다운로드
[Step 1500] Loss: 3.4676 | Accuracy: 12.50%
[Step 1600] Loss: 3.6722 | Accuracy: 37.50%
[Step 1700] Loss: 2.9659 | Accuracy: 37.50%
[Step 1800] Loss: 3.5100 | Accuracy: 25.00%
[Step 1900] Loss: 3.0206 | Accuracy: 37.50%
[Step 2000] Loss: 2.2781 | Accuracy: 25.00%
[Step 2100] Loss: 3.0021 | Accuracy: 37.50%
[Step 2200] Loss: 3.0121 | Accuracy: 37.50%
[Step 2300] Loss: 3.2845 | Accuracy: 0.00%
[Step 2400] Loss: 3.4987 | Accuracy: 0.00%
[Step 2500] Loss: 3.0503 | Accuracy: 25.00%
[Step 2600] Loss: 4.0142 | Accuracy: 25.00%
[Step 2700] Loss: 2.8773 | Accuracy: 25.00%
[Step 2800] Loss: 3.4030 | Accuracy: 12.50%
[Step 2900] Loss: 2.5390 | Accuracy: 25.00%
[Step 3000] Loss: 19569.1270 | Accuracy: 25.00%
[Step 3100] Loss: 3.0101 | Accuracy: 25.00%
[Step 3200] Loss: 2.9950 | Accuracy: 25.00%
[Step 3300] Loss: 3.1323 | Accuracy: 12.50%
[Step 3400] Loss: 4.1949 | Accuracy: 25.00%
[Step 3500] Loss: 2.5544 | Accuracy: 50.00%
[Step 3600] Loss: 2.5854 | Accuracy: 25.00%
[Step 3700] Loss: 2.7387 | Accuracy: 12.50%
[Step 3800] Loss: 2.8103 | Accuracy: 50.00%
[Step 3900] Loss: 3.0976 | Accuracy: 12.50%
[Step 4000] Loss: 3.0215 | Accuracy: 37.50%
[Step 4100] Loss: 1.8952 | Accuracy: 37.50%
[Step 4200] Loss: 1.7171 | Accuracy: 50.00%
[Step 4300] Loss: 2.8768 | Accuracy: 25.00%
[Step 4400] Loss: 1.8737 | Accuracy: 75.00%
[Step 4500] Loss: 2.9811 | Accuracy: 37.50%
[Step 4600] Loss: 2.3103 | Accuracy: 25.00%
[Step 4700] Loss: 2.3149 | Accuracy: 50.00%
[Step 4800] Loss: 1.2650 | Accuracy: 62.50%
[Step 4900] Loss: 3.7492 | Accuracy: 37.50%
-> Result: SoftDTW (Temporal Consistency): Loss=741.5049, Accuracy=30.75%

>>> Starting Experiment: Smoothing + SoftDTW
[Step 0000] Loss: 5.1303 | Accuracy: 12.50%
[Step 0100] Loss: 5.0574 | Accuracy: 0.00%
[Step 0200] Loss: 5.3846 | Accuracy: 0.00%
[Step 0300] Loss: 4.5871 | Accuracy: 12.50%
[Step 0400] Loss: 4.1574 | Accuracy: 25.00%
[Step 0500] Loss: 4.9851 | Accuracy: 0.00%
[Step 0600] Loss: 4.4097 | Accuracy: 12.50%
[Step 0700] Loss: 3.6791 | Accuracy: 12.50%
```

3. 결론

이번 실험을 통해 다음과 같은 결론을 도출할 수 있었습니다.

첫째, 모델의 성능은 충분한 학습량(Epoch) 확보가 전제되어야 제대로 평가할 수 있습니다.

둘째, Label Smoothing은 데이터셋의 특성(객체 중심 vs 행동 중심)에 구애받지 않고 모델의 과적합을 억제하여 범용적이고 안정적인 성능 향상을 이끌어냅니다.

셋째, SoftDTW와 같은 시계열 특화 손실 함수는 데이터셋이 시간적 순서를 강하게 요구할 때(SSv2) 비로소 효과를 보이기 시작합니다. 그러나 이를 극대화하기 위해서는 데이터셋의 난이도에 맞춘 정밀한 하이퍼파라미터(Margin, Weight 등...) 튜닝이 필요할 것 같습니다.

결론적으로, 목적에 맞는 적절한 손실 함수의 선택과 데이터셋 특성에 대한 이해가 비디오 분류 모델 파인튜닝의 핵심임을 알 수 있었습니다.